

Linux From Terminal To First Script

A fast reset using the existing CST2415 command-line, filesystem, and Bash materials.

Today

- Use the terminal without fear
- Navigate the filesystem
- Create and inspect files
- Run a small Bash script
- Submit command evidence plus reflection

Course source anchors

- Project 2-2: BASH shell basics
- Project 3-1: navigating Linux paths
- Chapter 3: directory structure
- Bash script docs: save, chmod, execute

Why Linux Matters For Software Work

Linux is not just an operating system topic. It is where a lot of technical work actually happens.

Practical jobs use Linux to

- Run servers and cloud workloads
- Package applications in containers
- Automate repetitive tasks
- Inspect logs, files, and processes
- Move and analyze text data

Beginner translation

- Terminal = control panel
- Files = raw material
- Scripts = repeatable instructions
- Output = evidence
- Errors = clues

The Mental Model

Keep this stack in your head while commands are flying by.

Application or script



Python or Bash



Shell command



Terminal window



Linux distribution, such as Fedora



Linux kernel and hardware

If students can explain this stack, they can orient themselves in most beginner Linux tasks.

First Commands: Identify The Session

Before changing anything, ask the system where you are and who you are.

```
$ whoami
```

```
$ hostname
```

```
$ date
```

```
$ pwd
```

```
$ ls
```

These are confidence commands: quick, safe, and useful.

Linux Is Case Sensitive

This comes directly from the Project 2-2 terminal exercise.

Works

```
$ date  
Mon May 4 ...
```

Does not work

```
$ Date  
bash: Date: command not found
```

date and Date are different names.

Directories: Absolute And Relative Paths

This is the heart of Project 3-1.

Absolute path

- Starts at /
- Works from anywhere
- Example: /etc
- Example: /home/student

Relative path

- Starts from your current folder
- Depends on where you are
- Example: ..
- Example: archive/notes.txt

Navigation Commands

The goal is not speed. The goal is knowing where you are before you run a command.

```
$ pwd
$ cd
$ cd ~
$ cd /
$ cd ..
$ cd ../../
$ cd /etc
$ cd ~/bootcamp
```

Use pwd after cd until students stop getting lost.

Files Are Data You Can Inspect

Chapter 3 emphasizes viewing file contents before editing or executing anything.

Create and view

- touch
- echo
- cat
- head
- tail
- less

Identify and search

- file
- grep
- grep -i
- find
- history
- man -k or apropos

Permissions: Read, Write, Execute

For today, keep permissions beginner-level.

```
$ ls -l  
-rw-r--r-- notes.txt  
-rwxr-xr-x hello.sh
```

r = read

w = write

x = execute

The script needs x permission if students run it as ./hello.sh.

First Script: Save, chmod, Execute

This compresses the course's first Bash script assignment into one working example.

```
$ nano hello.sh
```

```
#!/bin/bash  
echo "Hello from my script"  
date  
pwd
```

```
$ chmod +x hello.sh  
$ ./hello.sh
```

A script is a repeatable command sequence saved in a file.

Why Scripts Matter

Scripts turn terminal knowledge into repeatable work.

Manual work

- Type commands one at a time
- Easy to forget a step
- Hard to prove what happened
- Hard to repeat exactly

Scripted work

- Save the steps
- Run the same way again
- Share with someone else
- Use output as evidence

Today's Lab: Evidence Plus Explanation

Most of class is hands-on. Students submit proof and a short writing response.

Build

- ~/cst2415_lab1
- notes.txt
- commands.txt
- hello.sh
- system_snapshot.sh

Submit

- Terminal output or screenshots
- Contents of both scripts
- Output from running the scripts
- Short written answers

Lab Timing

Keep the lecture short so students actually practice.

- 0-20 min Lecture and live demo
- 20-30 min Start lab together
- 30-80 min Student lab work
- 80-95 min Writing/reflection
- 95-100 min Upload check and recap

If time runs short, prioritize terminal evidence plus reflection over perfect scripts.

Exit Ticket

Students should leave with a beginner mental model, not mastery.

Students should be able to say

- I can tell where I am in Linux
- I can create and read files
- I know absolute vs relative paths
- I can explain r, w, and x
- I ran a basic Bash script

Next class

- Use scripts for input and output
- Capture command output
- Start connecting scripts to app workflows
- Prepare for Python/API work